

Multi-Tenant SaaS Platform Backend Documentation

Database-per-tenant architecture with dynamic per-company schemas, a Django + DRF backend, and a superadmin control plane.

Django 6.1 · DRF 3.18 · MySQL/MariaDB · djangorestframework-simplejwt
Generated for the `tenant-architecture` project

Table of Contents

- [1](#) The Core Idea
- [2](#) Project Layout & App Responsibilities
- [3](#) Request Lifecycle
- [4](#) Database Routing (TenantRouter)
- [5](#) Dynamic Models — Why the ORM Needs Help
- [6](#) Authentication — Two Kinds of Login, One JWT Authenticator
- [7](#) Authorization — Cross-Tenant Superadmin Access
- [8](#) Tenant Onboarding, End to End
- [9](#) Migrations — One Command, Not Two
- [10](#) Field-Level Dynamic Schema
- [11](#) Central Admin API Surface
- [12](#) Full Local Setup

1. The Core Idea

This is a **database-per-tenant** multi-tenant platform. Every company ("tenant") that signs up gets:

- A row in a central registry (company name, slug, owner, logo, status).
- Its own **physical MySQL database** (e.g. `tenant_tata`, `tenant_moca`).
- Its own `Employee` / `Customer` tables, whose *columns can differ per company* — Tata might have 8 employee fields, another company 14, both drawn from the same superadmin-controlled master catalog.

A single Django process serves every tenant. Which physical database a request hits is decided **per request, at runtime**, based on the company slug in the URL or JWT.

	Central DB (<code>default</code>)	Tenant DB (<code>tenant_<slug></code>)
Holds	<code>Tenant</code> , <code>TenantModule</code> , <code>FieldCatalog</code> , <code>TenantFieldConfig</code> , <code>SuperAdminUser</code> , <code>TenantAuditLog</code> , Django admin's own <code>auth_user</code>	<code>Employee</code> , <code>Customer</code> , and that company's own, fully isolated <code>auth_user</code> table
Who writes	Superadmin actions, provisioning	That tenant's own users, tenant-scoped API calls
Created	Once, up front	Dynamically, one per company, at onboarding time

Key mental model: there is no per-tenant deployment. One running Django process, one codebase, many MySQL connections — the request decides which one to use.

2. Project Layout & App Responsibilities

```
backend/  
  config/          settings.py, urls.py – wiring only  
  tenants/         the control plane (central DB)  
  core_auth/       authentication (two flavors – see §6)  
  modules/  
    employees/     tenant-scoped Employee CRUD  
    customers/     tenant-scoped Customer CRUD
```

tenants/ — the brain of the system

File	Responsibility
<code>models.py</code>	<code>Tenant</code> , <code>TenantModule</code> , <code>FieldCatalog</code> , <code>TenantFieldConfig</code> , <code>SuperAdminUser</code> , <code>TenantAuditLog</code>
<code>middleware.py</code>	<code>TenantResolverMiddleware</code> — resolves which tenant a request is for
<code>context.py</code>	a <code>contextvar</code> holding “which DB alias is active right now”
<code>routers.py</code>	<code>TenantRouter</code> — the actual DB-routing decision
<code>db_registry.py</code>	injects a tenant's DB connection into Django at runtime
<code>provisioning.py</code>	the onboarding pipeline: create DB → migrate → sync schema
<code>schema_sync.py</code>	adds real columns to a tenant's tables via raw <code>ALTER TABLE</code>
<code>dynamic_models.py</code>	builds a Python model class matching those columns, so the ORM can use them
<code>dynamic_fields.py</code>	shared data-type → Django-field mapping used by both of the above
<code>mixins.py</code>	shared <code>schema</code> action + superadmin audit-logging for viewsets
<code>views.py</code>	the whole <code>/api/superadmin/...</code> surface
<code>audit.py</code>	writes <code>TenantAuditLog</code> rows for cross-tenant superadmin writes
<code>management/commands/migrate.py</code>	extends Django's own <code>migrate</code> with <code>--tenant=<slug></code>

modules/employees/, modules/customers/

Deliberately minimal models — only `id`, `created_at`, `updated_at`. Every company-specific column (name, department, PAN number, whatever fields Superadmin enabled) is added *dynamically*, not hardcoded. `app_label` stays `employees` / `customers` even though the Python path is `modules.employees` / `modules.customers` — Django derives the label from the last dotted path segment, which is why moving these apps into the `modules/` package required no migration changes.

core_auth/

File	Responsibility
<code>views.py</code>	<code>SuperAdminLoginView</code> , <code>TenantLoginView</code>
<code>authentication.py</code>	<code>TenantJWTAuthentication</code> — handles the superadmin token's missing <code>user_id</code>
<code>permissions.py</code>	<code>IsTenantUserOrSuperAdmin</code> — role + tenant-slug matching
<code>urls.py</code> / <code>urls_tenant_scoped.py</code>	central auth routes vs. per-tenant-slug auth routes

3. Request Lifecycle

```
GET /api/tata/employees/
|
▼
TenantResolverMiddleware
  • pulls slug "tata" from the URL (or X-Tenant header)
  • Tenant.objects.get(slug="tata") - 404/403 if missing/suspended
  • set_current_tenant(tenant, "tata") → contextvar
  • request.tenant = tenant
  |
  ▼
TenantJWTAuthentication (validates the JWT - see §6)
  |
  ▼
IsTenantUserOrSuperAdmin (checks role - see §7)
  |
  ▼
EmployeeViewSet
  get_queryset() → get_dynamic_model('employee', tenant).objects.using('tata')
  get_serializer_class() → ModelSerializer built off that same dynamic model
  |
  ▼
TenantRouter.db_for_read/write
  → app_label 'employees' is tenant-scoped → routes to the 'tata' MySQL connection
  |
  ▼
response, then middleware's `finally:` → clear_current_tenant()
```

Routes exempt from tenant resolution (`/__superadmin`, `/api/superadmin`, `/admin`, `/swagger`, `/redoc`) go straight through — they only ever touch the central DB.

4. Database Routing (TenantRouter)

```
class TenantRouter:
    def _tenant_alias(self, model):
        app_label = model._meta.app_label
        if app_label in settings.TENANT_APPS:          # employees, customers
            return get_current_tenant_db() or 'default'
        if app_label in settings.TENANT_SHARED_APPS:  # auth, contenttypes
            return get_current_tenant_db() # None → Django's normal 'default' resolution
        return None

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        if app_label in settings.TENANT_APPS:
            return db != 'default'
        if app_label in settings.TENANT_SHARED_APPS:
            return True # both 'default' (Django admin) and every tenant alias
        return db == 'default'
```

`TENANT_SHARED_APPS = ['auth', 'contenttypes']` is the reason tenant users can log in at all: `auth.User` migrates onto **both** the central DB (for Django admin / the central superuser) **and** every tenant DB (so each company has its own, completely separate `auth_user` table). The router sends `auth` queries to whichever tenant is active in the current request's context, falling back to `default` when no tenant context is set.

5. Dynamic Models — Why the ORM Needs Help

Django's ORM can only touch columns that are *declared* on a Python model class — it can't discover “whatever happens to be in the table.” But every tenant's `Employee` table has a different column set, decided by which `FieldCatalog` fields that tenant has enabled. So the work is split in two:

`schema_sync.py` — the physical schema

```
def sync_tenant_schema(tenant):
    connection = connections[tenant.slug]
    configs = tenant.field_configs.select_related('field').filter(enabled=True)
    with connection.schema_editor() as schema_editor:
        for cfg in configs:
            if _column_exists(connection, model._meta.db_table, cfg.field.field_key):
                continue
            field = build_model_field(cfg.field.data_type)
            field.set_attributes_from_name(cfg.field.field_key)
            schema_editor.add_field(model, field)  # real ALTER TABLE ADD COLUMN
```

This runs a real `ALTER TABLE ... ADD COLUMN` directly against the tenant's MySQL database — **not** a Django migration file. With dozens of tenants each with a different column set, one migration file per tenant per field change doesn't scale. The “schema” for Employee/Customer is data (`TenantFieldConfig` rows); `sync_tenant_schema` is what makes the real table match that data. Disabling a field leaves its column in place (soft-hide at the serializer layer) rather than dropping data.

`dynamic_models.py` — the ORM-facing schema

```
def get_dynamic_model(entity, tenant):
    configs = tenant.field_configs.filter(enabled=True, field__entity=entity)
    attrs = {'created_at': ..., 'updated_at': ...}
    for cfg in configs:
        attrs[cfg.field.field_key] = build_model_field(cfg.field.data_type)
    attrs['Meta'] = type('Meta', (), {
        'db_table': static_model._meta.db_table,  # same physical table
        'app_label': _APP_LABEL[entity],
        'managed': False,  # schema_sync.py owns the schema, not `migrate`
        'ordering': ['id'],
    })
    return type(f'Employee_{tenant.slug}_{hash(...)}', (models.Model,), attrs)
```

This builds (and caches) a small Model subclass that declares exactly a tenant's enabled columns, pointed at the same physical table the static `Employee` model manages. `EmployeeViewSet` queries through *this* class, so `ModelSerializer` derives the correct field type (boolean, integer, date, email...) automatically — no manual per-data-type serializer code needed, and list/retrieve/create/update/delete all work exactly like a normal Django app.

The static `modules/employees/models.py:Employee` class only ever declares `id` / `created_at` / `updated_at`. It's the thing that gets migrated via `manage.py migrate`, but it is **never** used directly for CRUD — the dynamic subclass is.

6. Authentication — Two Kinds of Login, One JWT Authenticator

Flow	Checks against	Token contents
Superadmin <code>SuperAdminLoginView</code>	<code>tenants.models.SuperAdminUser</code> (central DB, hashed password)	<code>role: superadmin</code> , <code>username</code> — no <code>user_id</code> (not a Django <code>auth.User</code>)
Tenant user <code>TenantLoginView</code>	<code>request.tenant</code> 's own database (real Django <code>auth.User</code>)	<code>role: tenant_user</code> , <code>tenant_slug</code> , plus a normal <code>user_id</code>

DRF's stock `JWTAuthentication` would reject the superadmin token outright — it requires a `user_id` claim to look up a real user. `core_auth/authentication.py:TenantJWTAuthentication` overrides `get_user()`:

```
class TenantJWTAuthentication(JWTAuthentication):
    def get_user(self, validated_token):
        if validated_token.get('role') == 'superadmin':
            return SuperAdminPrincipal(validated_token.get('username', 'superadmin'))
        return super().get_user(validated_token) # normal path for tenant users
```

Login/public endpoints (`SuperAdminLoginView`, `TenantLoginView`, `TenantPublicInfoView`) set `authentication_classes = []` explicitly — otherwise a stale/expired Bearer token already in the request would make DRF reject the request *before* the login logic even runs. Access tokens currently live **6 hours** (`SIMPLE_JWT.ACCESS_TOKEN_LIFETIME`); refresh tokens live 7 days.

7. Authorization — Cross-Tenant Superadmin Access

`core_auth/permissions.py:IsTenantUserOrSuperAdmin` (used by both `EmployeeViewSet` and `CustomerViewSet`):

```
def has_permission(self, request, view):
    role = request.auth.get('role')
    if role == 'superadmin':
        return True # any slug
    if role == 'tenant_user':
        return request.auth.get('tenant_slug') == request.tenant.slug # only its own
    return False
```

A superadmin token passes this check for every `/api/<slug>/...` regardless of slug — that's the entire mechanism behind "superadmin can open any company's data." A tenant user's token only ever passes for the one `tenant_slug` baked into it at login.

Whatever writes a superadmin makes get logged: `tenants/mixins.py:TenantEntityViewSetMixin._maybe_audit()` checks `request.auth.get('role')` and, if `superadmin`, writes a `TenantAuditLog` row recording the actor, the action (`create_employee`, `update_customer`, ...), and the affected row's id. Tenant users' own writes are *not* logged this way — only cross-tenant superadmin edits are.

A separate, simpler permission — `tenants/views.py:IsSuperAdmin` — gates the entire `/api/superadmin/...` surface (tenant registry, field catalog, per-tenant field-config, per-tenant user management). It checks purely `request.auth.get('role') == 'superadmin'`, deliberately not any Django `is_staff` / `is_superuser` flag, since those don't apply to the superadmin stand-in principal at all.

8. Tenant Onboarding, End to End

`tenants/provisioning.py:provision_tenant()`, triggered by `POST /api/superadmin/tenants/`:

1. **Register.** Creates a `Tenant` row (central DB) with a generated `db_name` (`tenant_<slug>`).
2. **Create the physical database.**

```
cursor.execute(
    f'CREATE DATABASE IF NOT EXISTS `{db_name}` '
    f'CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci'
)
```

Run on the central connection, whose DB user must have `CREATE` privileges. `db_name` always comes from a validated `SlugField`, so there's no SQL-injection risk in building this string directly.

3. **Register the connection.** `register_tenant_database()` injects the new alias into `settings.DATABASES` / `connections.databases` live — no process restart needed.
4. **Run tenant migrations.** `call_command('migrate', tenant=slug)` — creates `employees_employee`, `customers_customer`, `auth_user`, etc. inside the new database, with just the core columns.
5. **Enable selected modules.** Creates a `TenantModule` row per selected module key.
6. **Sync field schema.** `sync_tenant_schema(tenant)` — applies whatever fields are already configured for that tenant (usually none yet, until Superadmin configures them).

On process startup, `TenantsConfig.ready()` calls `load_all_tenant_databases()`, reloading every existing active tenant's connection back into `settings.DATABASES` — so after a server restart, all previously provisioned tenant DBs are immediately reachable again with no re-provisioning.

9. Migrations — One Command, Not Two

`tenants/management/commands/migrate.py` subclasses Django's own `migrate` command and adds a `--tenant` flag — there's no separate command name to remember:

```
python manage.py migrate           # central DB — stock Django, unchanged
python manage.py migrate --tenant=tata  # every tenant app, onto tata's DB
python manage.py migrate employees --tenant=tata  # just one app
python manage.py migrate_all_tenants  # loops migrate --tenant= over every active tenant
```

Internally: looks up the `Tenant` row, calls `register_tenant_database()`, then re-invokes stock `migrate` once per app in `TENANT_SHARED_APPS + TENANT_APPS`. Passing an app label outside that set (e.g. `tenants`, a central-only app) raises a clear `CommandError` rather than silently doing nothing.

Situation	Command
First-ever setup of the project	<code>migrate</code> (creates central tables)
Add a field to the <code>Tenant</code> model	<code>makemigrations tenants</code> → <code>migrate</code>
Add a field to the <code>Employee</code> model that every company should get	<code>makemigrations employees</code> → <code>migrate_all_tenants</code>
A brand-new company signs up	Nothing manual — <code>provision_tenant()</code> calls it automatically
Superadmin changes which fields a company has enabled	Nothing manual — not a migration at all, it's <code>sync_tenant_schema()</code>

10. Field-Level Dynamic Schema

Problem: different companies need different fields on the same logical table — but all valid fields are predefined and controlled by Superadmin, not freely typed by tenants.

1. **Master Field Catalog** (central DB): `FieldCatalog` defines every field Superadmin has made available per entity type — `entity` (employee/customer), `field_key`, `label`, `data_type` (string/integer/date/boolean/enum/email/text), `is_required_default`, `options`.
2. **Tenant Field Selection:** `TenantFieldConfig` maps `(tenant, field) → enabled, is_required, order`. Superadmin picks which subset of the catalog applies to each tenant.
3. **Applying the selection:** `sync_tenant_schema()` compares a tenant's enabled `TenantFieldConfig` rows against the actual table columns and adds whatever's missing.
4. **Exposing it to the frontend:** `GET /api/<slug>/employees/schema/` and `.../customers/schema/` (added by `TenantEntityViewSetMixin.schema`) return `[{key, label, data_type, required, options}, ...]` for that tenant's enabled fields, so the UI can render forms/tables without hardcoding a single field name.

11. Central Admin API Surface

All on `tenants/views.py:TenantViewSet`, one `ModelViewSet` with several actions:

Endpoint	Purpose
<code>GET/POST /api/superadmin/tenants/</code>	List companies / onboard a new one (calls <code>provision_tenant</code>)
<code>POST /api/superadmin/tenants/<id>/suspend/</code>	Sets <code>Tenant.status = suspended</code>
<code>POST /api/superadmin/tenants/<id>/reactivate/</code>	Sets <code>Tenant.status = active</code>
<code>GET/POST /api/superadmin/tenants/<id>/field-config/</code>	Merge full catalog with this tenant's selection; <code>POST</code> upserts <code>TenantFieldConfig</code> and immediately runs <code>sync_tenant_schema</code>
<code>GET/POST /api/superadmin/tenants/<id>/users/</code>	List usernames / create a login user directly in that tenant's own <code>auth_user</code> table (hashed via <code>make_password</code>)
<code>GET/POST /api/superadmin/field-catalog/</code>	<code>FieldCatalogViewSet</code> — manage the master field list itself

Full tenant-facing URL map:

```
/admin/           Django admin (central DB only)
/swagger/, /redoc/  API docs

/api/auth/superadmin/login/  Superadmin login (central DB)
/api/superadmin/tenants/     Tenant registry CRUD (superadmin only)
/api/superadmin/field-catalog/  Manage the master FieldCatalog

/api//auth/login/           Tenant user login
/api//public-info/         Public branding (name/logo) – no auth
/api//employees/           Tenant-scoped Employee CRUD
/api//employees/schema/     That tenant's enabled Employee fields
/api//customers/           Tenant-scoped Customer CRUD
/api//customers/schema/     That tenant's enabled Customer fields
```

12. Full Local Setup

```
# 1. Central database
mysql -u root -p -e \
  "CREATE DATABASE tenant_platform_central CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"

# 2. Backend
cd backend
uv venv
uv sync
cp .env.example .env
uv run python manage.py migrate
uv run python manage.py createsuperuser      # Django admin login
uv run python manage.py runserver

# 3. Seed a superadmin (tenant-platform superadmin, not Django admin)
uv run python manage.py shell
>>> from django.contrib.auth.hashers import make_password
>>> from tenants.models import SuperAdminUser
>>> SuperAdminUser.objects.create(username="admin", password_hash=make_password("choose-a-real-password"))

# 4. Onboard a company via the API/UI, then give it a login (see §11: users endpoint),
#    then configure its fields (see §11: field-config endpoint) – that's the whole loop.
```

That's the whole system: middleware picks the tenant → auth identifies who's asking → permissions check what they're allowed to touch → the dynamic model/router pair make the ORM work against a schema that's different for every company → provisioning and the extended `migrate` command are what create and evolve those per-company schemas in the first place.